AI-Programming-Assignment-5

Dhimanth Reddy
SE24UCSE055

PART 1: SEARCH ALGORITHMS

Aim

To implement and test the following AI search algorithms:

1. Minimax Search
2. Alpha-Beta Pruning
3. Heuristic Alpha-Beta Search
4. Monte Carlo Tree Search (MCTS)

Description

Minimax is used to find the best move in a two-player game.

Alpha-Beta pruning improves Minimax by removing branches that do not affect the final decision.

Heuristic Alpha-Beta uses an evaluation function and depth limit to reduce computation.

Monte Carlo Tree Search uses random simulations to estimate the best move.

Observation

Alpha-Beta pruning explored fewer nodes than Minimax while producing the same optimal move. MCTS also produced good moves using simulations.

PART 2: AI TRAVEL PLANNER

Aim

To design an AI based travel planner using existing knowledge about tourist destinations, food preferences, budget and activities.

Description

The travel planner recommends destinations based on user interests, season, budget and food preferences. It also generates a simple tour plan and estimated travel cost.

Output

User Profile 1

```
PS C:\Users\dhima> python -u "c:\Users\dhima\OneDrive\Desktop\AI assignment 4\# travel_planner.py"
===== AI Travel Planner =====
Available interests: ['adventure', 'history', 'beach', 'shopping', 'nature']
Available food prefs: ['vegetarian', 'seafood lover', 'street food', 'fine dining']

--- User Profile 1: Beach lover, seafood, winter, budget $80/day ---
Recommended destinations: ['Goa', 'Cape Town']

===== Tour Plan: Goa, India =====
Duration     : 5 days
Est. Budget  : ~$300 USD ($60/day)
Best season  : winter
Local cuisine: Seafood, Vindaloo, Bebinca
Languages    : Konkani, English

Day-by-Day Plan:
  Day 1:
    Morning  : Visit Calangute Beach
    Afternoon: Explore local Seafood cuisine
    Evening  : Local cultural experience
  Day 2:
    Morning  : Visit Dudhsagar Falls
    Afternoon: Explore local Vindaloo cuisine
    Evening  : Free time / shopping
  Day 3:
    Morning  : Visit Old Goa Churches
    Afternoon: Explore local Bebinca cuisine
    Evening  : Local cultural experience
  Day 4:
    Morning  : Visit Calangute Beach
    Afternoon: Explore local Seafood cuisine
    Evening  : Free time / shopping
  Day 5:
    Morning  : Visit Dudhsagar Falls
    Afternoon: Explore local Vindaloo cuisine
    Evening  : Local cultural experience
```

```
Total Estimated Cost: ~$300 USD (excluding flights)
```

User Profile 2

```
--- User Profile 2: History buff, fine dining, fall, budget $300/day ---
Recommended destinations: ['Paris', 'New York', 'Kyoto', 'Goa', 'Cape Town']

===== Tour Plan: Paris, France =====
Duration     : 7 days
Est. Budget  : ~$1400 USD ($200/day)
Best season  : spring, fall
Local cuisine: French, Crepes, Wine
Languages    : French

Day-by-Day Plan:
  Day 1:
    Morning  : Visit Eiffel Tower
    Afternoon: Explore local French cuisine
    Evening  : Local cultural experience
  Day 2:
    Morning  : Visit Louvre Museum
    Afternoon: Explore local Crepes cuisine
    Evening  : Free time / shopping
  Day 3:
    Morning  : Visit Notre-Dame Cathedral
    Afternoon: Explore local Wine cuisine
    Evening  : Local cultural experience
  Day 4:
    Morning  : Visit Eiffel Tower
    Afternoon: Explore local French cuisine
    Evening  : Free time / shopping
  Day 5:
    Morning  : Visit Louvre Museum
    Afternoon: Explore local Crepes cuisine
    Evening  : Local cultural experience
  Day 6:
    Morning  : Visit Notre-Dame Cathedral
    Afternoon: Explore local Wine cuisine
    Evening  : Free time / shopping
  Day 7:
    Morning  : Visit Eiffel Tower
    Afternoon: Explore local French cuisine
    Evening  : Local cultural experience

Total Estimated Cost: ~$1400 USD (excluding flights)
```

Observation

The system successfully generated travel recommendations and personalized tour plans based on user preferences.

PART 3: KNOWLEDGE GRAPHS

Aim

To understand Knowledge Graphs and explore tools used to build them.

Description

A Knowledge Graph stores information in the form of triples:

(Subject, Predicate, Object)

Example:

(Paris, locatedIn, France)

The implementation demonstrates a simple travel-related Knowledge Graph and query operations.

Tools Explored

1. RDFLib
2. Neo4j
3. Protégé
4. Apache Jena
5. Wikidata

Output

Knowledge Graph Triples

```
PS C:\Users\dhima> python -u "c:\Users\dhima\OneDrive\Desktop\AI assignment 4\# knowledge_graphs.py"
===== Knowledge Graph Demo =====

Knowledge Graph Triples:
  (Paris, isA, City)
  (Paris, locatedIn, France)
  (Paris, locatedIn, Europe)
  (Paris, hasAttraction, EiffelTower)
  (Paris, hasAttraction, Louvre)
  (EiffelTower, isA, Landmark)
  (EiffelTower, builtIn, 1889)
  (Kyoto, isA, City)
  (Kyoto, locatedIn, Japan)
  (Kyoto, locatedIn, Asia)
  (Kyoto, hasAttraction, FushimiInari)
  (FushimiInari, isA, Temple)
  (Crepes, isA, Food)
  (Crepes, originatesFrom, France)
  (Paris, hasCuisine, Crepes)
  (Matcha, isA, Food)
  (Kyoto, hasCuisine, Matcha)
  (Paris, avgDailyCostUSD, 200)
  (Kyoto, avgDailyCostUSD, 150)
```

Query Results

```
--- Query: All attractions in Paris ---
  ('Paris', 'hasAttraction', 'EiffelTower')
  ('Paris', 'hasAttraction', 'Louvre')

--- Query: What is FushimiInari? ---
  ('FushimiInari', 'isA', 'Temple')

--- Query: What cities are in Europe? ---
  Paris is located in Europe

--- Query: All foods and their origins ---
  Crepes originates from France

===== Equivalent RDFLib code =====

# This is how the same KG would be built with RDFLib:

from rdflib import Graph, Literal, URIRef, Namespace
from rdflib.namespace import RDF, RDFS

EX = Namespace("http://example.org/travel#")
g = Graph()

g.add((EX.Paris, RDF.type, EX.City))
g.add((EX.Paris, EX.locatedIn, EX.France))
g.add((EX.Paris, EX.hasAttraction, EX.EiffelTower))
g.add((EX.EiffelTower, RDF.type, EX.Landmark))

# SPARQL query
result = g.query('''
    SELECT ?city ?attraction WHERE {
        ?city <http://example.org/travel#hasAttraction> ?attraction .
    }
''')
for row in result:
    print(row)

# Save as Turtle format
g.serialize("travel_kg.ttl", format="turtle")


Tools summary:
Tool            Type            Features                          Access
--------------------------------------------------------------------------------
RDFLib          Python library  SPARQL queries, RDF/OWL           pip install rdflib
Neo4j           Graph DB        Cypher queries, visualization     neo4j.com
Protege         GUI editor      OWL ontologies, reasoning         protege.stanford.edu
Apache Jena     Java framework  Inference, SPARQL                 jena.apache.org
Wikidata        Open KG         Public knowledge, SPARQL          query.wikidata.org
PS C:\Users\dhima>
```

Observation

Knowledge Graphs allow structured storage of information and support efficient querying and reasoning.

PART 4: BAYESIAN NETWORKS

Aim

To study Bayesian Networks and perform probabilistic inference.

Description

A Bayesian Network was created to model student performance using the following variables:

- Study
- Difficulty
- Grade
- Pass

Conditional Probability Tables (CPTs) were used to represent relationships between variables.

Output

Probability Calculations

Joint Distribution / Sanity Check

```
PS C:\Users\dhima> python -u "c:\Users\dhima\OneDrive\Desktop\AI assignment 4\# bayesian_network.py"
===== Bayesian Network: Student Exam Model =====

Network: Study -> Grade <- Difficult -> ... Grade -> Pass

P(Pass = True)                  = 0.5892
P(Pass=True | Study=True)       = 0.7180
P(Pass=True | Study=False)      = 0.3960
P(Pass=True | Study=T, Diff=T)  = 0.5500
P(Pass=True | Study=T, Diff=F)  = 0.8300

Sanity check: P(Pass=T) + P(Pass=F) = 1.0000  (should be 1.0)
P(Grade = Good)                 = 0.5560
P(Difficult=T | Grade=Bad)      = 0.5946

--- Interpretation ---
Studying increases pass probability as expected.
Hard exams reduce grade quality.
Knowing the grade gives info about difficulty (explaining away).

===== Full Joint Distribution =====
Study   Diff    Grade   Pass    Prob
--------------------------------------
True    True    True    True    0.108000
True    True    True    False   0.012000
True    True    False   True    0.024000
True    True    False   False   0.096000
True    False   True    True    0.291600
True    False   True    False   0.032400
True    False   False   True    0.007200
True    False   False   False   0.028800
False   True    True    True    0.014400
False   True    True    False   0.001600
False   True    False   True    0.028800
False   True    False   False   0.115200
False   False   True    True    0.086400
False   False   True    False   0.009600
False   False   False   True    0.028800
False   False   False   False   0.115200
Total                           1.000000  (should be 1.0)
PS C:\Users\dhima>
```

Observation

The Bayesian Network correctly performed probability calculations and inference based on the given evidence.

CONCLUSION

The assignment successfully implemented important Artificial Intelligence concepts including search algorithms, travel planning systems, knowledge graphs and Bayesian networks. The programs were tested using multiple test cases and produced correct outputs.